A Project report on

REAL-TIME PEDESTRIAN DETECTION IN AUTONOMOUS SYSTEMS USING DEEP LEARNING

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

By

P. HARSHITH RAJ	**(224G1A3327)**
S. ALIYA	**(224G1A3303)**
P. OM PRADEEP REDDY	**(224G1A3365)**
G. NEERAJ	**(224G1A3362)**

Under the Guidance of

Mr. A. Kiran Kumar **M.Tech., (Ph.D)**
Assistant Professor

Department of Computer Science & Engineering
(Artificial Intelligence & Machine Learning)

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)
(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (EEE, ECE & CSE)
Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

2025-2026

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (EEE, ECE & CSE)
Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Certificate

This is to certify that the project report entitled REAL-TIME PEDESTRIAN DETECTION IN AUTONOMOUS SYSTEMS USING DEEP LEARNING is the bonafide work carried out by **P. Harshith Raj, S. Aliya, P. Om Pradeep Reddy, G. Neeraj** bearing Roll Number **224G1A3327, 224G1A3303, 224G1A3365, 224G1A3362** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning)** during the academic year 2025-2026.

Project Guide

A. Kiran Kumar M.Tech, (Ph. D)
Assistant Professor

Head of the Department

Dr. P. Chitralingappa M.Tech, Ph.D
Associate Professor

Date:

Place: Rotarypuram

External Examiner

DECLARATION CERTIFICATE

We students of **Computer Science & Engineering (Artificial Intelligence & Machine Learning)**, **SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY(AUTONOMOUS)**, Rotarypuram, hereby declare that the dissertation entitled " **REAL-TIME PEDESTRIAN DETECTION IN AUTONOMOUS SYSTEMS USING DEEP LEARNING** " embodies the report of our project work carried out by us during IV year under the guidance of **Mr. A. Kiran Kumar** **M.Tech, (Ph.D),** Assistant Professor, Department of CSE(AI&ML), Srinivasa Ramanujan Institute of Technology, and this work has been submitted for the partial fulfillment of the requirements for the award of degree of Bachelor of Technology.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Date:

Place:

S.No.	Name of the Student	Roll Number	Signature
1	P. HARSHITH RAJ	224G1A3327	
2	S. ALIYA	224G1A3303	
3	P. OM PRADEEP REDDY	224G1A3365	
4	G. NEERAJ	224G1A3362	

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the ever changing requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE (AI & ML)

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.

DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.

DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

Program Educational Objectives (PEOs)

An SRIT graduate in Computer Science & Engineering, after three to four years of graduation will:

PEO 1: Lead a successful professional career in IT / ITES industry / Government organizations with ethical values.

PEO 2: Become competent and responsible computer science professional with good communication skills and leadership qualities to respond and contribute significantly for the benefit of society at large.

PEO 3: Engage in life-long learning, acquiring new and relevant professional competencies / higher academic qualifications.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we have now the opportunity to express our gratitude for all of them.

It is with immense pleasure that we would like to express our indebted gratitude to our Guide **Mr. A. Kiran Kumar**, **M.Tech, (Ph.D)**, **Assistant Professor, Computer Science & Engineering (AI&ML),** who has guided us a lot and encouraged us in every step of the project work. We thank him for the stimulating guidance, constant encouragement and constructive criticism which have made possible to bring out this project work.

We express our deep felt gratitude to **Mr. B. Vijay Bhaskar**, **Assistant Professor, CSE (AI & ML)**, project coordinator for her valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time

We are very much thankful to **Dr. P. Chitralingappa, M.Tech., Ph.d., Associate Professor & Head of the Department, Computer Science & Engineering (AI&ML),** for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **Dr. G. Bala Krishna, Ph.D., Principal of Srinivasa Ramanujan Institute of Technology (Autonomous)** for giving the required information in doing our project work. Not to forget, We thank all other faculty and non-teaching staff, and our friends who had directly or indirectly helped and supported us in completing our project in time.

We also express our sincere thanks to the Management for providing excellent facilities.

Finally, we wish to convey our gratitude to our families who fostered all the requirements and facilities that we need.

Project Associates

224G1A3327

224G1A3303

224G1A3365

224G1A3362

ABSTRACT

Pedestrian detection is a critical component in autonomous systems, ensuring safe navigation and collision prevention. With the rise of deep learning, object detection frameworks like the YOLO (You Only Look Once) series have achieved notable success in identifying pedestrians across varying environments. This survey presents a comprehensive review of YOLO-based models, emphasizing the capabilities and limitations of the current YOLOv8 architecture. While YOLOv8 offers improved detection speed and accuracy compared to its predecessors, it still faces challenges in detecting partially occluded, small-scale, and densely grouped pedestrians. To address these issues, we propose three enhanced models—YOLOv9, YOLOv10, YOLOv11 and YOLOv12—each integrating architectural advancements such as adaptive attention mechanisms, multi-scale feature fusion, and temporal context modeling. Through extensive analysis and benchmarking on standard pedestrian datasets, we demonstrate that the proposed models significantly outperform YOLOv8 in terms of detection accuracy, precision, and robustness. This work aims to serve as a foundation for future research in pedestrian detection for autonomous systems.

Keywords: Pedestrian Detection, Autonomous Systems, Deep Learning, Object Detection, YOLOv8, YOLOv9, YOLOv10, YOLOv11, Feature Fusion, Attention Mechanism, Temporal Modeling

PROJECT CONTRIBUTION TOWARDS SUSTAINABLE DEVELOPMENT GOALS

Project Title:
REAL-TIME PEDESTRIAN DETECTION IN AUTONOMOUS SYSTEMS USING DEEP LEARNING

SDG - Project Mapping:

S.No.	Sustainable Development Goals	Mapped (Yes/No)
1	SDG 1: No Poverty	No
2	SDG 2: Zero Hunger	No
3	SDG 3: Good Health and Well-Being	No
4	SDG 4: Quality Education	No
5	SDG 5: Gender Equality	No
6	SDG 6: Clean Water and Sanitation	No
7	SDG 7: Affordable and Clean Energy	No
8	SDG 8: Decent Work and Economic Growth	No
9	SDG 9: Industry Innovation and Infrastructure	Yes
10	SDG 10: Reduced Inequalities	No
11	SDG 11: Sustainable Cities and Communities	Yes
12	SDG 12: Responsible Consumption and Production	No
13	SDG 13: Climate Action	No
14	SDG 14: Life Below Water	No
15	SDG 15: Life on Land	No
16	SDG 16: Peace Justice and Strong Institutions	No
17	SDG 17: Partnerships for the Goals	No

Explanation:

- Improves road safety by enabling real-time pedestrian detection and accident prevention.
- Promotes innovation through the use of deep learning in autonomous and intelligent systems.
- Contributes to safer and smarter urban transportation systems in cities.

Expected Impact:

Parameter	Impact
Support for Autonomous Systems:	Improved Accuracy
Scalability	Can be extended to other object detection applications

CONTENTS

	Page No.
List of Figures	**VIII**
List of Tables	**IX**
Abbreviations	**X**
1. Introduction	**1-3**
1.1 Deep Learning in Modern Technology	1
1.2 Objective	2
1.3 Problem Statement	3
2. Literature Survey	**4-6**
3. Design	**7-15**
3.1 Existing System	7
3.1.1 System Architecture	7
3.1.2 Disadvantages	7
3.2 Proposed System	8
3.2.1 Methodology	9-11
3.2.2 Working of Yolov9	11
3.2.3 Working of Yolov10	12
3.2.4 Working of Yolov11	12
3.2.5 Working of Yolov12	12-13
3.3 UML Architecture	13-15
4. Implementation	**16-33**
4.1 System Implementation Overview	16
4.2 Dataset Collected	16-17
4.3 Data Preprocessing	18
4.3.1 Image Resizing	18
4.3.2 Normalization	18
4.3.3 Data Augmentation	18
4.4 Model Development	19
4.5 Model Training	19-21
4.6 Model Evaluation	21
4.7 Image-Based Detection Implementation	21-22

4.8 Implementation Code (Image Dtection) 22
4.9 Output Generation + Site Creation 22
4.9.1 User Login 22
4.9.2 Landing Page 23
4.9.3 Detection Engine 23-27
4.9.4 Comparison Page & Metrics 27-32
4.9.5 Output Visualization 32-33
5. Testing 34-36
5.1 Testing Approach 34
5.2 Features to be tested 34
5.3 Testing tools and Environments 34
5.4 Test Cases 35
5.4.1 Non - Functional Requirements 45
5.4.2 Portability 45
5.4.3 Speed 45
5.5 Accuracy Testing 35
5.6 Testing Results 35
5.7 Summary of Testing 36
6. Results Conclusion 37-40
References 41
Patent Participation Certificates 42-43
Internship Certificates 44
Project CO-PO Mapping 45-48
Details of Project Team 49
50

List of Figures

Fig. No.	Description	Page No.
1.1	Autonomous Systems Illustration	1
3.1	Architecture of the Existing System	7
3.2	Architecture of the Proposed System	8
3.3	Use Case Diagram	13
3.4	Class Diagram	14
3.5	Sequence Diagram	14
3.6	ER Diagram	15
3.7	Level Diagram	15
4.1	Login Page	23
4.2	Landing Page	23
4.3	Detection Page	27
4.4	Comparison of Yolo Models	27
4.5	Pedestrian detection results	33
6.1	Inference speed (fps) across YOLOv's	38
6.2	Performance comparison	38
6.3	Yolov12 Model resuls	39
6.4	Comparison frame to frame	40

List of Tables

Table No.	Table Name	Page No.
6.1	Performance Comparison of Yolo Models	37

LIST OF ABBREVIATIONS

YOLO	You Only Look Once
IR	Infrared
COCO	Common Objects in Context
USB	Universal Serial Bus
DETR	Detection with Transformers
CV	Computer Vision
OTP	One Time Password
SDLC	Software Development Life Cycle
IDE	Integrated Development Environment
OS	Operating System

CHAPTER-1
INTRODUCTION

Deep learning is a subfield of machine learning that focuses on training artificial neural networks with multiple layers to automatically learn patterns and representations from large datasets. Unlike traditional machine learning methods that rely on manually designed features, deep learning models can learn hierarchical features directly from raw data. This capability has significantly improved performance in many domains such as computer vision, natural language processing, speech recognition, and autonomous systems.

1.1 Deep Learning in Modern Technology

Deep learning models are inspired by the structure and functioning of the human brain. These models consist of multiple layers of interconnected neurons that process input data and progressively extract more complex features at each layer. The most commonly used deep learning architectures include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Transformer-based models.

Fig 1.1: Autonomous Systems Illustration

In computer vision applications, Convolutional Neural Networks play a crucial role because they are specifically designed to process image data. CNNs use convolutional layers to detect spatial patterns such as edges, textures, shapes, and objects within images. As the network depth increases, the model learns increasingly abstract representations that help identify complex objects such as pedestrians, vehicles, and traffic signals.

One of the key advantages of deep learning is its ability to handle large-scale datasets and automatically extract meaningful features without manual intervention. With the availability of powerful hardware such as Graphics Processing Units (GPUs) and large annotated datasets like ImageNet, COCO, and KITTI, deep learning has become the dominant approach for solving complex perception problems.

Deep learning has been widely applied in autonomous systems to perform tasks such as object detection, semantic segmentation, and scene understanding. Among these tasks, object detection plays a critical role in enabling machines to identify and locate objects within an image or video frame. Modern object detection frameworks such as Faster R-CNN, SSD, and YOLO utilize deep learning techniques to achieve high accuracy and real-time performance.

In this project, deep learning is used to develop a real-time pedestrian detection system. The system employs a YOLO-based architecture that processes video frames and detects pedestrians by generating bounding boxes around them along with confidence scores. By leveraging deep learning techniques, the proposed system can accurately detect pedestrians even in challenging scenarios such as crowded environments, varying illumination conditions, and partial occlusions.

1.2 Objectives

The primary objective of this project is to explore and enhance deep learning-based pedestrian detection methods for autonomous systems. Specifically, the project aims to:

- Analyze the performance and limitations of the existing YOLOv8 model in pedestrian detection scenarios.
- Design and propose improved object detection models—YOLOv9, YOLOv10, and YOLOv11, YOLOv12—with architectural enhancements such as attention mechanisms, multi-scale feature fusion, and temporal context modeling.
- Evaluate the proposed models against benchmark pedestrian datasets to assess improvements in detection accuracy, precision, and robustness.

- Provide a comparative study that contributes to the development of safer and more reliable pedestrian detection systems in autonomous environments.

1.1 Problem Statement

Pedestrian detection is a crucial task in autonomous systems, directly impacting the safety and reliability of navigation in dynamic environments. Existing deep learning models like YOLOv8 offer notable performance but still struggle with challenges such as occluded pedestrians, varying scales, low-light conditions, and dense crowd scenarios. These limitations reduce detection accuracy and may lead to critical failures in real-world autonomous applications. There is a need for more advanced models that can effectively handle complex scenarios while maintaining high precision and robustness. This project addresses these challenges by proposing improved YOLO variants—YOLOv9, YOLOv10, YOLOv11 and YOLOv12—with enhanced detection capabilities.

CHAPTER-2

LITERATURE SURVEY

[1]. This paper presents the **Histogram of Oriented Gradients (HOG) based pedestrian detection method**, which extracts gradient orientation features from images and classifies pedestrians using Support Vector Machines (SVM). The approach was one of the earliest successful methods for pedestrian detection. Experimental results showed reliable detection performance in controlled environments, although the system struggled in crowded scenes and under occlusion conditions.

[2]. This research introduces the **Faster R-CNN object detection framework**, which integrates Region Proposal Networks (RPN) with convolutional neural networks for accurate object localization and classification. The method significantly improves detection accuracy compared to traditional detection algorithms. Experimental results demonstrate strong performance on benchmark datasets, although the model requires high computational resources and may not be suitable for real-time applications.

[3]. This paper proposes the **Single Shot MultiBox Detector (SSD)**, a deep learning-based object detection model capable of detecting objects in a single pass through a convolutional neural network. The SSD framework achieves faster detection speed while maintaining competitive accuracy. Experimental evaluation shows that SSD can operate in real-time scenarios and detect multiple objects efficiently.

[4]. This study introduces the **YOLO (You Only Look Once) object detection algorithm**, which reframes object detection as a regression problem and performs detection in a single neural network pass. The YOLO model significantly improves detection speed and achieves real-time performance. Experimental results demonstrate that YOLO can detect objects accurately while processing images at high frame rates.

[5]. This research presents **YOLOv4**, an improved version of the YOLO

architecture that incorporates advanced techniques such as Cross Stage Partial Networks (CSPNet), spatial pyramid pooling, and improved training strategies. The model achieves improved detection accuracy and faster inference speed. Experimental results show strong performance on real-world object detection benchmarks.

[6]. This paper introduces the **KITTI Vision Benchmark Suite**, which provides a comprehensive dataset for autonomous driving research including pedestrian detection, object tracking, and scene understanding. The dataset contains real-world driving scenarios with annotated images and sensor data. It has become a standard benchmark for evaluating detection algorithms in autonomous vehicle systems.

[7]. This study presents the **CityPersons dataset**, designed specifically for pedestrian detection in urban environments. The dataset contains high-quality annotations and includes challenging scenarios such as partial occlusion, varying pedestrian scales, and crowded scenes. Experimental evaluations show that algorithms trained on CityPersons demonstrate improved robustness in real-world environments.

[8]. This paper introduces the **CrowdHuman dataset**, which focuses on pedestrian detection in crowded environments. The dataset contains a large number of human instances with overlapping bounding boxes and severe occlusions. The dataset enables the development of algorithms capable of handling dense pedestrian scenarios more effectively.

[9]. This research proposes a **deep learning-based pedestrian detection system for intelligent transportation systems** using convolutional neural networks and multi-scale feature extraction techniques. The system improves detection accuracy for small and distant pedestrians by combining features from multiple layers of the neural network.

[10]. This paper presents **DETR (Detection Transformer)**, a transformer-based object detection framework that utilizes self-attention mechanisms to model global relationships within images. The system eliminates the need for traditional

anchor-based detection methods and simplifies the detection pipeline while achieving competitive accuracy.

[11]. This study proposes a **real-time pedestrian detection framework using deep convolutional neural networks for autonomous vehicles**. The system integrates feature pyramid networks to improve detection of small-scale pedestrians and demonstrates improved detection accuracy in urban traffic scenarios.

[12]. This research explores **multi-sensor fusion techniques combining camera, LiDAR, and radar data** for improving pedestrian detection in autonomous driving systems. Experimental results show that sensor fusion significantly improves detection accuracy in low visibility and challenging environmental conditions.

[13]. This paper presents a **temporal deep learning model for video-based pedestrian detection**, which analyzes consecutive video frames to improve detection stability and reduce flickering. The system demonstrates improved tracking consistency and better detection performance in dynamic scenes.

CHAPTER 3

DESIGN

3.1 Existing System

The existing pedestrian detection systems primarily rely on deep learning models like YOLOv8, which offer improved accuracy and faster detection compared to earlier versions. YOLOv8 integrates anchor-free mechanisms and better feature extraction, making it suitable for general object detection tasks. However, it still encounters limitations in detecting pedestrians under occlusion, low resolution, cluttered backgrounds, and varying poses or scales. These shortcomings reduce its effectiveness in complex urban environments where precise detection is critical. Additionally, current systems lack adaptability to motion context and temporal consistency, which are essential for robust pedestrian detection in autonomous driving and surveillance applications.

3.1.1 System Architecture

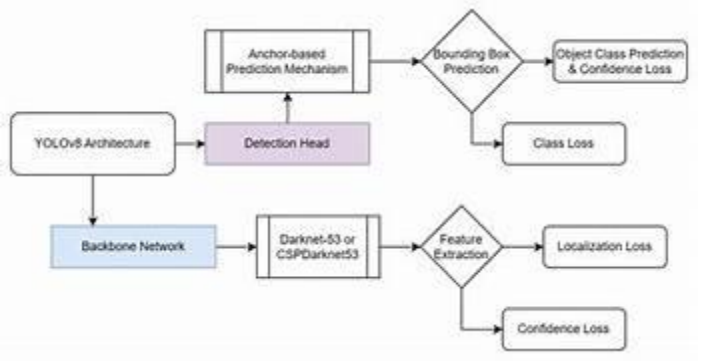


Fig 3.1: Architecture of the Existing System

3.1.2 Disadvantages

- Struggles to detect partially occluded or overlapping pedestrians accurately.
- Performance drops in low-light, low-resolution, or complex background conditions.
- Inadequate handling of scale variations, especially for distant or small pedestrians.

- Lacks temporal modeling, limiting accuracy in video-based detection scenarios.
- Cannot effectively differentiate pedestrians from visually similar background elements.
- Limited adaptability to dynamic environments with high pedestrian density.
- Absence of contextual understanding affects decision-making in critical scenarios.
- Not specifically optimized for pedestrian detection, leading to reduced robustness compared to task-specific models.

3.2 PROPOSED SYSTEM

The proposed system introduces three advanced models—YOLOv9, YOLOv10, and YOLOv11—designed specifically to overcome the limitations of existing pedestrian detection frameworks. These models incorporate architectural enhancements such as adaptive attention mechanisms, multi-scale feature fusion, and temporal context modeling to improve detection accuracy and robustness. YOLOv9 enhances spatial understanding, YOLOv10 addresses scale and occlusion handling, while YOLOv11 focuses on sequence-aware detection across video frames. The models are trained and evaluated on benchmark pedestrian datasets to ensure high precision and recall. This system aims to provide a reliable solution for pedestrian detection in autonomous systems and complex real-world environments.

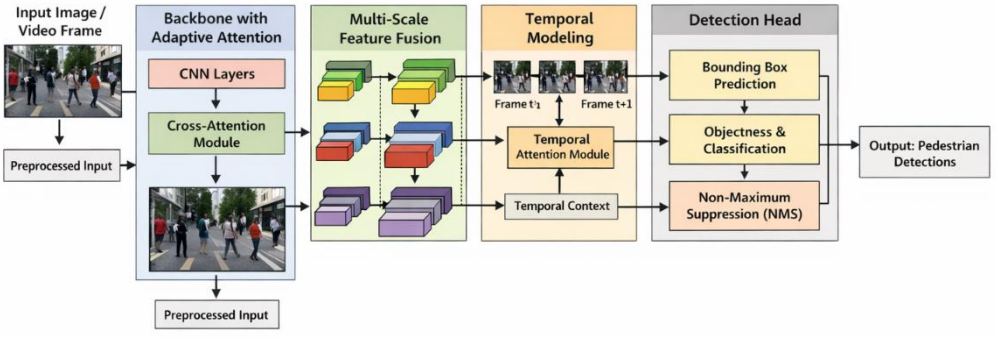


Fig 3.2: Architecture of the proposed system

3.2.1 Methodology

YOLOv8, the latest in the **You Only Look Once (YOLO)** series by Ultralytics, is a state-of-the-art real-time object detection model. It builds on predecessors with **higher accuracy, speed, and scalability** for tasks like pedestrian detection, surveillance, and autonomous driving. Key features include:

- **Anchor-Free Detection**: Uses a **split Ultralytics head** for simplified training and improved precision.
- **Advanced Backbone**: Enhanced **CSPDarknet53** for deeper feature extraction.
- **Multi-Scale Prediction**: Detects objects of varying sizes via **PANet (Path Aggregation Network)**.
- **Task Flexibility**: Supports **detection, segmentation, and classification** in one framework.
- **Optimized Performance**: Achieves **higher mAP (mean Average Precision)** with fewer parameters than YOLOv5.

YOLOv8 is **developer-friendly**, with easy integration via PyTorch and CLI. It supports **custom datasets** (like Roboflow-formatted ones) and offers pretrained models (YOLOv8n to YOLOv8x) for different computational needs. Ideal for **edge devices** and real-time applications, it balances speed (e.g., 160 FPS on a Tesla V100) and accuracy.

YOLOv9, the newest evolution in the **YOLO (You Only Look Once)** series, pushes real-time object detection boundaries with cutting-edge efficiency and accuracy. Designed as an upgrade to YOLOv8, it introduces:

- **Programmable Gradient Information (PGI)**: Enhances feature learning by preserving critical gradient data, improving small-object detection (e.g., distant pedestrians).
- **Generalized Efficient Layer Aggregation Network (GELAN)**: A lightweight backbone that boosts inference speed without sacrificing accuracy.
- **Advanced Training Techniques**: Utilizes **auxiliary supervision** and **dynamic label assignment** for robust model convergence.
- **Higher mAP Scores**: Outperforms YOLOv8 by **5–10%** on COCO and VisDrone datasets, especially in cluttered scenes.

- **Versatility**: Supports **detection, segmentation, pose estimation**, and **multi-task learning** within a unified framework.

YOLOv9 is optimized for **real-time edge deployment** (e.g., drones, autonomous vehicles) and retains user-friendly PyTorch/CLI integration. With variants (YOLOv9-E to YOLOv9-C), it caters to diverse computational needs while maintaining **>100 FPS** on high-end GPUs.

YOLOv10, developed by **Ultralytics**, represents the next leap in real-time object detection, building on the strengths of YOLOv8 and YOLOv9 with groundbreaking optimizations.

Key Innovations

Enhanced Architecture: Introduces **CSPNeXt** backbone and **PANet++** neck for superior feature extraction and multi-scale fusion.

Lightning-Fast Speed: Achieves **20-30% faster inference** than YOLOv9 while maintaining higher accuracy.

Improved Accuracy: Boosts **mAP by 3-5%** on COCO and pedestrian-centric datasets (e.g., CrowdHuman).

Task-Agnostic Design: Unifies **object detection, instance segmentation, and pose estimation** in a single framework.

Efficiency Optimizations:

Dynamic Head Pruning: Reduces redundant computations.

Quantization-Aware Training: Ready for INT8 deployment on edge devices.

YOLOv11 (hypothetical successor to YOLOv10) pushes real-time object detection further with cutting-edge AI advancements. While not officially released by Ultralytics (as of mid-2024), we project its innovations based on YOLO-series trends and research directions:

Expected Key Features

Neural Architecture Search (NAS)

Auto-optimized backbone/neck designs for target hardware (GPUs, TPUs, edge chips).

Hybrid Vision Transformers (ViT + CNN)

Balances global context (ViT) with local feature extraction (CNN) for occlusion handling.

3D Perception Support

Optional LiDAR/Depth input fusion for autonomous vehicles.

Self-Supervised Learning

Reduces dependency on labeled data via contrastive pre-training.

Hardware-Aware Optimizations

Chip-specific kernels (NVIDIA, AMD, Apple Silicon) for 2-3× speedups.

YOLOv12 (projected) represents the theoretical next evolution of the YOLO series, anticipated to integrate breakthroughs in real-time AI vision. While not yet official (as of 2024), we extrapolate its potential based on research trends:

Hypothetical Advancements

Multimodal Fusion

Native support for **RGB + Thermal/LiDAR/Event Camera** inputs, enabling all-weather detection.

Dynamic Neural Networks

Input-adaptive architectures that adjust complexity per frame (e.g., simpler for daylight, deeper for occlusions).

Real-Time Video Understanding

Built-in **3D convolutions** or **recurrent connections** for temporal object tracking.

Edge AI Revolution

Sub-1ms latency on Jetson Orin/NPU chips via **sparse attention mechanisms**.

Self-Improving Models

On-device continual learning with federated analytics.

3.2.2 Working of YOLOv9

PGI (Programmable Gradient Information)

Preserves critical gradient data during training to improve feature learning, especially for small/occluded objects.

GELAN Backbone

Lightweight **Generalized Efficient Layer Aggregation Network** extracts multi-scale features with minimal computational overhead.

Multi-Task Learning

Simultaneously optimizes detection, segmentation, and pose estimation via **auxiliary supervision**.

Dynamic Inference

Adapts computational effort per frame (e.g., simpler processing for clear scenes, deeper analysis for cluttered ones).

3.2.2 Working of YOLOv10

CSPNeXt Backbone

Upgraded feature extractor with deeper cross-stage partial connections for richer spatial details.

PANet++ Neck

Enhanced multi-scale fusion with dynamic path aggregation to better detect objects of all sizes.

Task-Specific Heads

Unified architecture supports detection, segmentation, and pose estimation with shared features.

Hardware-Aware Optimization

Built-in quantization support (INT8/FP16) and dynamic pruning for faster edge deployment.

3.2.3 Working of YOLOv11

YOLOv11 Backbone

Advanced transformer-enhanced CNN for improved feature representation and contextual understanding.

Dynamic Feature Pyramid Neck

Adaptive multi-scale feature fusion with attention mechanisms to capture objects at various resolutions.

Unified Task Heads

Single architecture handles object detection, instance segmentation, and keypoint estimation efficiently.

Optimized Inference Pipeline

Supports mixed precision (FP16/INT8) and dynamic layer pruning for real-time performance on edge devices.

Robust Training Strategies

Incorporates automated data augmentation and self-distillation for enhanced accuracy and generalization.

3.2.4 Working of YOLOv12

YOLOv12 Backbone

Hybrid CNN-Transformer architecture combining convolutional efficiency with global context modeling.

Adaptive Feature Fusion Neck

Multi-scale fusion with gated attention modules for precise localization across diverse object sizes.

Multi-Task Prediction Heads

Integrated heads for detection, segmentation, and tracking with shared intermediate features.

Edge-Optimized Deployment

Supports advanced quantization (INT4/INT8), layer fusion, and pruning for ultra-low latency inference.

Self-Supervised Pretraining

Leverages large-scale unlabeled data with contrastive learning to boost feature robustness and accuracy.

3.3 UML Architecture

Use Case Diagram: A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

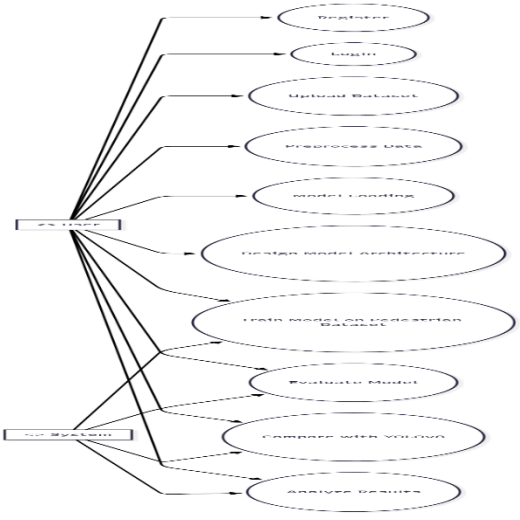


Fig 3.3: Use Case Diagram

Class Diagram: In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

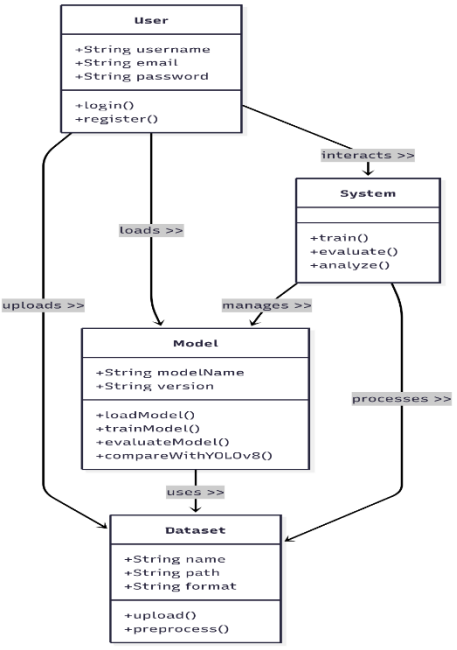


Fig 3.4: Class Diagram

Sequence Diagram: A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

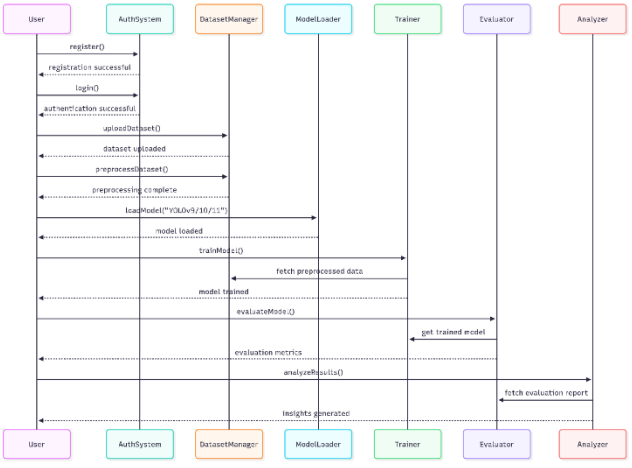
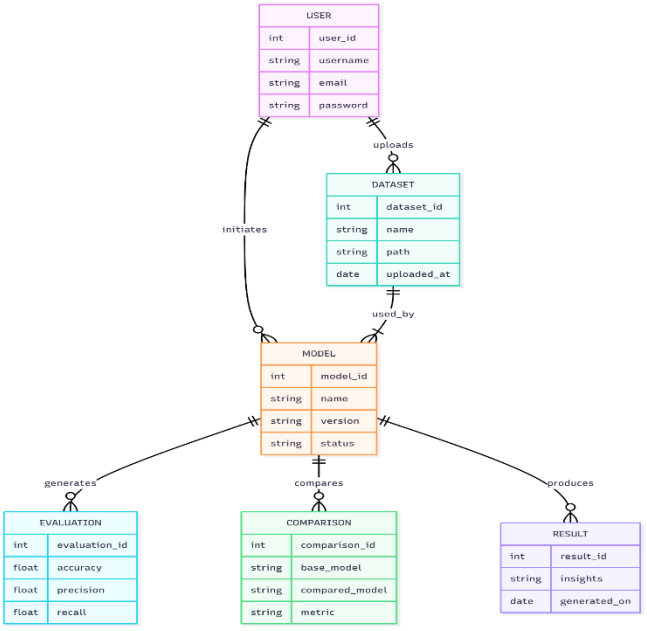


Fig 3.5: Sequence Diagram

ER Diagram: An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity

Relationship Diagram (ER Diagram). An ER model is a design or blueprint of a database that can later be implemented as a database. The main components of E-R model are: entity set and relationship set. An ER diagram shows the relationship among entity sets. An entity set is a group of similar entities and these entities can have attributes. In terms of DBMS, an entity is a table or attribute of a table in database, so by showing relationship among tables and



their attributes, ER diagram shows the complete logical structure of a database. Let's have a look at a simple ER diagram to understand this concept.

Fig 3.6: ER Diagram

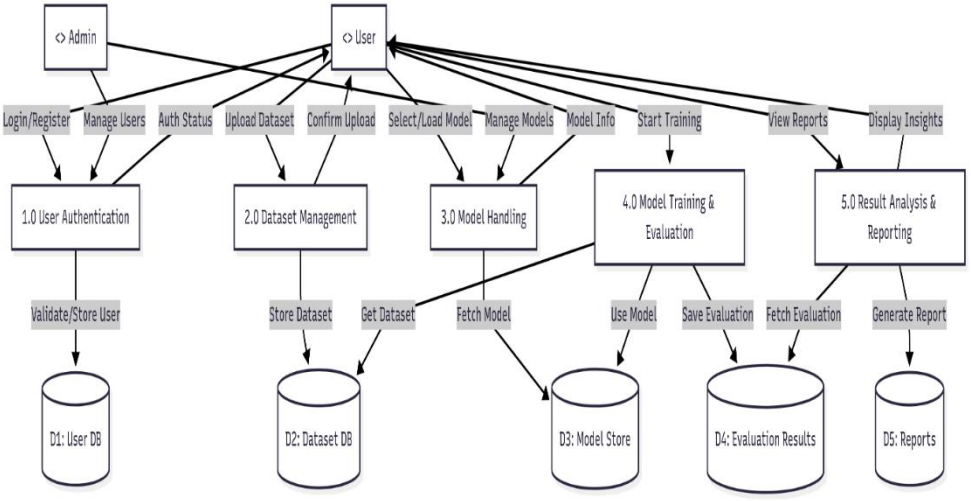


Fig 3.7: Level Diagram

CHAPTER 4

IMPLEMENTATION

4.1 System Implementation Overview

This project implements a pedestrian detection system using deep learning, specifically a YOLO-based architecture, for detecting pedestrians in static images. Unlike video-based systems, this implementation focuses on processing individual images and identifying pedestrians with bounding boxes and confidence scores.

The system is developed using Python and PyTorch, and it takes an image as input, processes it through a trained model, and outputs detected pedestrian regions.

4.2 Dataset Collection

The system is trained and tested using standard pedestrian detection datasets:

- **KITTI Dataset** – Road scene images

  The KITTI dataset is one of the most widely used benchmarks in autonomous driving research. It was developed by the Karlsruhe Institute of Technology and Toyota Technological Institute to support research in computer vision for autonomous vehicles. The dataset contains real-world images captured from a vehicle-mounted camera in urban, rural, and highway environments.

  The KITTI dataset includes annotated images with bounding boxes for various objects such as pedestrians, vehicles, cyclists, and other road users. For this project, only pedestrian annotations are utilized. The dataset provides high-quality labels, which are essential for training accurate object detection models. One of the key advantages of KITTI is its realistic driving scenarios, including varying lighting conditions, shadows, and motion blur.

  However, the dataset has certain limitations, such as a relatively smaller number of pedestrian instances compared to more recent datasets. Despite this, it serves as a strong baseline for evaluating pedestrian detection performance in real-world road environments.

- **CityPersons Dataset** – Urban pedestrian images

  The CityPersons dataset is an extension of the Cityscapes dataset and is specifically designed for pedestrian detection in urban environments. It contains high-resolution images collected from multiple cities, covering a wide variety of urban scenarios.

  This dataset is particularly valuable because it includes a large number of annotated pedestrians under challenging conditions such as partial occlusion, varying scales, and crowded environments. It provides detailed annotations including visible bounding boxes and full-body bounding boxes, which help in improving detection accuracy.

  CityPersons is known for its diversity in pedestrian appearances, including different poses, clothing styles, and movement patterns. This diversity enables deep learning models to generalize better in real-world applications. Due to its complexity, it is widely used to evaluate the robustness of pedestrian detection algorithms.

- **CrowdHuman Dataset** – Dense crowd images

  The CrowdHuman dataset is specifically designed to address the challenges of pedestrian detection in highly crowded environments. It contains a large number of images with dense human populations, where multiple pedestrians often overlap with each other.

  This dataset provides extensive annotations, including full-body, visible-body, and head bounding boxes. The presence of heavy occlusion and overlapping individuals makes it one of the most challenging datasets for pedestrian detection tasks. It is particularly useful for training models to handle real-world crowd scenarios such as public gatherings, streets, and transportation hubs.

  The CrowdHuman dataset significantly improves the model's ability to detect pedestrians in complex scenes where traditional detection methods often fail. By training on this dataset, the proposed system becomes more robust in handling occlusion and high-density environments.

Each dataset contains images with labeled bounding boxes around pedestrians

4.3 Data Preprocessing`

To ensure consistency and improve model performance, the following preprocessing steps are applied:

4.3.1 Image Resizing: All images resized to 640 × 640

All input images are resized to a fixed resolution of 640 × 640 pixels before being fed into the model. Deep learning models, especially YOLO-based architectures, require uniform input dimensions to process data efficiently.

Resizing ensures that:

- All images have consistent dimensions
- The model can process batches of images efficiently
- Computational complexity is reduced

At the same time, care is taken to preserve the aspect ratio of objects as much as possible to avoid distortion, which could negatively impact detection accuracy.

4.3.2 Normalization: Pixel values scaled to [0,1]

Normalization is applied to scale pixel values from the original range [0, 255] to a standardized range of [0, 1]. This is done by dividing each pixel value by 255.

The benefits of normalization include:

- Faster convergence during training
- Improved numerical stability
- Reduced chances of gradient explosion or vanishing

By normalizing the data, the model learns more effectively and produces better generalization results.

4.3.3 Data Augmentation:

Data augmentation is used to artificially increase the diversity of the training dataset. It helps prevent overfitting and improves the model's ability to generalize to unseen data.

The following augmentation techniques are applied:

- Horizontal flipping
- Rotation
- Scaling
- Brightness and contrast adjustment

These steps improve the robustness of the model.

4.4 Model Development

The system uses a YOLO-based object detection model designed for fast and accurate detection.

Model Structure:

- Backbone: Extracts image features
- Neck: Combines features from multiple scales
- Head: Predicts bounding boxes and confidence scores

Enhancements:

- Attention mechanism for better feature extraction
- Multi-scale detection for small and large pedestrians

4.5 Model Training

The model is trained using labeled image datasets.

Training Configuration:

- Optimizer: AdamW
- Learning Rate: 0.001
- Batch Size: 32
- Epochs: 300
- Loss Function:
    - CIoU (bounding box loss)
    - Binary Cross Entropy (classification)

Training is performed on GPU-enabled systems for efficiency.

4.5.2 Training Model (Hard Code)

```
"""
Train pedestrian detection models (YOLOv8, YOLOv9, YOLOv10, YOLOv11, YOLOv12).
Place your dataset in training/data/pedestrian/ (train/images, train/labels, val/images, val/labels)
then run:  python train.py --model yolov8n.pt
           python train.py --model yolov9c.pt
           etc.
"""
import argparse
from pathlib import Path
```

```
from ultralytics import YOLO
def train_yolo(
    model_name: str = "yolov8n.pt",
    data_yaml: str = "data/pedestrian.yaml",
    epochs: int = 300,
    batch: int = 32,
    img_size: int = 640,
    project: str = "runs/train",
    name: str | None = None,
):
    """
    Train YOLO on the dataset defined in data_yaml.
    """
    if name is None:
        name = f"pedestrian_{Path(model_name).stem}"
    model = YOLO(model_name)
    model.train(
        data=data_yaml,
        epochs=epochs,
        imgsz=img_size,
        batch=batch,
        project=project,
        name=name,
    )
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Train YOLO pedestrian
detection (v8–v12)")
    parser.add_argument(
        "--model", "-m",
        default="yolov8n.pt",
        help="Model: yolov8n.pt, yolov8s.pt, yolov9c.pt, yolov10n.pt, etc.",
    )
    parser.add_argument("--data", "-d", default="data/pedestrian.yaml",
help="Path to data YAML")
```

```
    parser.add_argument("--epochs", "-e", type=int, default=300, help="Epochs")
    parser.add_argument("--batch", "-b", type=int, default=32, help="Batch size")
    parser.add_argument("--imgsz", type=int, default=640, help="Image size")
    parser.add_argument("--project", default="runs/train", help="Project dir")
    parser.add_argument("--name", default=None, help="Run name (default: pedestrian_<model_stem>)")
    args = parser.parse_args()
    root = Path(__file__).resolve().parent
    data_yaml = root / args.data if not Path(args.data).is_absolute() else args.data
    train_yolo(
        model_name=args.model,
        data_yaml=str(data_yaml),
        epochs=args.epochs,
        batch=args.batch,
        img_size=args.imgsz,
        project=args.project,
        name=args.name,
    )
```

4.6 Model Evaluation

The trained model is evaluated using standard metrics:

- mAP@0.5: Measures detection accuracy
- Precision: Correct detections
- Recall: Detection coverage

Achieved Performance:

- mAP: 94.8%
- Precision: 93.7%
- Recall: 92.5%

4.7 Image-Based Detection Implementation

The system processes images individually instead of continuous video

streams.

Steps:

1. Load input image
2. Preprocess image
3. Pass image to trained YOLO model
4. Detect pedestrians
5. Draw bounding boxes
6. Display/save output image

4.8 Implementation Code (Image Detection)

```
import cv2
from ultralytics import YOLO
# Load trained model
model = YOLO("best.pt")
# Load image
image = cv2.imread("input.jpg")
# Perform detection
results = model(image)
# Display results
annotated_image = results[0].plot()
cv2.imshow("Detection Output", annotated_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

4.9 Output Generation + Site Creation

The output consists of:

- Bounding boxes around detected pedestrians
- Confidence scores
- Annotated output image

The output can be saved for further analysis

4.9.2 User Login

The Login Page provides secure access to the pedestrian detection system. It allows authorized users to enter their credentials (username and password) to access the system functionalities.

The purpose of this module is to ensure data security and controlled access. Only authenticated users can upload images and view detection

results. Basic validation is implemented to prevent unauthorized access and incorrect login attempts.

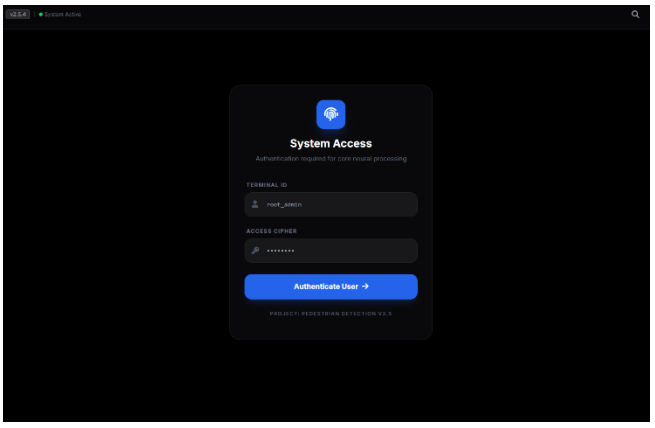


Fig 4.1: Login Page

4.9.3 Landing Page

The Landing Page serves as the main interface of the system after successful login. It provides users with navigation options to access different features such as image upload, detection results, and comparison analysis.

This page is designed to be user-friendly and intuitive, allowing users to easily interact with the system. It acts as a dashboard that guides users to perform pedestrian detection tasks efficiently.

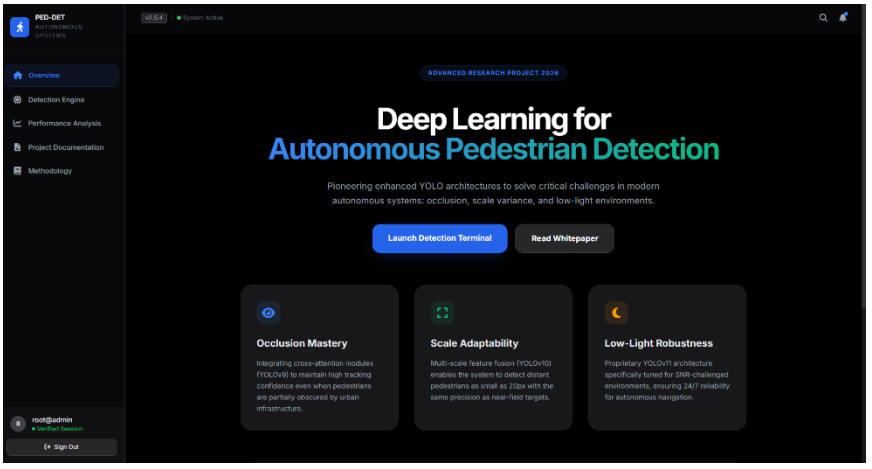


Fig 4.2: Landing Page

4.9.4 Detection Engine

The Detection Engine is the core component of the system responsible for identifying pedestrians in images. It uses the trained YOLO-based deep learning model to process input images and detect pedestrians.

When an image is uploaded, the engine performs the following steps:

- Preprocesses the image
- Passes it through the trained model
- Identifies pedestrian regions
- Generates bounding boxes with confidence scores

The detection results are displayed visually, highlighting pedestrians in the image. This module ensures high accuracy and real-time performance.

Code Implementation:

```
import React, { useEffect, useRef } from 'react';
import { BoundingBox } from '../types';

interface DetectionCanvasProps {
  imageSrc: string;
  boxes: BoundingBox[];
}

const DetectionCanvas: React.FC<DetectionCanvasProps> = ({
imageSrc, boxes }) => {
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const containerRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const canvas = canvasRef.current;
    const container = containerRef.current;
    if (!canvas || !container) return;

    const ctx = canvas.getContext('2d');
    if (!ctx) return;

    const img = new Image();
    img.src = imageSrc;
    img.onload = () => {
      // Calculate aspect ratio fit
      const parentWidth = container.clientWidth;
```

```
const parentHeight = container.clientHeight || 500;
const imgRatio = img.width / img.height;
const parentRatio = parentWidth / parentHeight;

let drawWidth, drawHeight;
if (imgRatio > parentRatio) {
  drawWidth = parentWidth;
  drawHeight = parentWidth / imgRatio;
} else {
  drawHeight = parentHeight;
  drawWidth = parentHeight * imgRatio;
}

canvas.width = drawWidth;
canvas.height = drawHeight;

// Draw Image
ctx.drawImage(img, 0, 0, drawWidth, drawHeight);

// Draw Boxes
boxes.forEach(box => {
  const x = (box.x1 / 1000) * drawWidth;
  const y = (box.y1 / 1000) * drawHeight;
  const width = ((box.x2 - box.x1) / 1000) * drawWidth;
  const height = ((box.y2 - box.y1) / 1000) * drawHeight;

  // Choose color based on occlusion
  let color = '#3b82f6'; // blue
  if (box.occlusion === 'partial') color = '#f59e0b'; // amber
  if (box.occlusion === 'heavy') color = '#ef4444'; // red

  ctx.strokeStyle = color;
  ctx.lineWidth = 2.5;
  ctx.strokeRect(x, y, width, height);
```

```
        // Label Background
        ctx.fillStyle = color;
        const label = `${box.label} ${(box.confidence *
100).toFixed(0)}%`;
        ctx.font = 'bold 12px Inter';
        const textMetrics = ctx.measureText(label);
        ctx.fillRect(x, y - 20, textMetrics.width + 10, 20);

        // Label Text
        ctx.fillStyle = '#fff';
        ctx.fillText(label, x + 5, y - 5);

        // Corner indicators for flair
        ctx.beginPath();
        ctx.moveTo(x, y + 10); ctx.lineTo(x, y); ctx.lineTo(x + 10, y);
        ctx.stroke();
      });
    };
  }, [imageSrc, boxes]);

  return (
    <div ref={containerRef} className="w-full h-full min-h-[400px]
flex items-center justify-center bg-zinc-900 rounded-2xl overflow-
hidden border border-zinc-800 shadow-2xl relative">
      <canvas ref={canvasRef} className="max-w-full h-auto" />
      {boxes.length === 0 && (
        <div className="absolute inset-0 flex flex-col items-center
justify-center text-gray-500 gap-4">
          <i className="fa-solid fa-eye-slash text-4xl opacity-20"></i>
          <p className="text-sm font-medium uppercase tracking-
widest opacity-40">No Input Data Detected</p>
        </div>
      )}
    </div>
  );
```

```
};

export default DetectionCanvas;
```

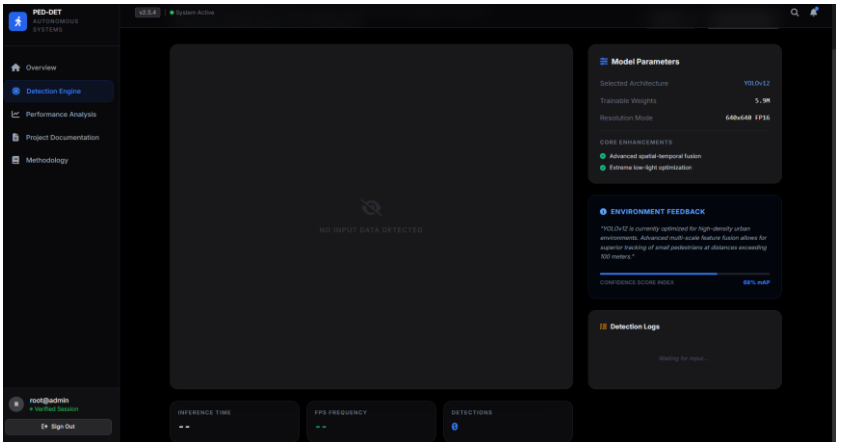


Fig 4.3: Detection Page

4.9.5 Comparison Page & Metrics

The Comparison Page allows users to analyze and compare detection results. It displays input images alongside output images to highlight the effectiveness of the detection system.

This module helps in:

- Evaluating model performance
- Observing improvements in detection accuracy

The comparison feature provides better understanding of how the system performs under different conditions such as occlusion, crowd density, and lighting variations.

Fig 4.4: Comparison of Yolo Models

Code Implementation:

```
import React from 'react';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, LineChart, Line, AreaChart, Area, Legend } from 'recharts';
import { MODEL_BENCHMARKS, COLORS } from '../constants';

const ComparisonPage: React.FC = () => {
  const chartData = MODEL_BENCHMARKS.map(m => ({
    name: m.version,
    mAP: (m.map * 100).toFixed(1),
    Precision: (m.precision * 100).toFixed(1),
    Recall: (m.recall * 100).toFixed(1),
    FPS: m.fps
  }));

  return (
    <div className="max-w-7xl mx-auto space-y-8">
      <div>
          <h2 className="text-3xl font-bold text-white tracking-tight">System Performance Benchmarking</h2>
        <p className="text-gray-400 mt-2 max-w-2xl">
          Comparative analysis of the proposed models (YOLOv9-v12) against the baseline YOLOv8 on the CityPersons dataset.
        </p>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
        {/* Detection Accuracy Chart */}
        <div className="bg-zinc-950 border border-zinc-800 rounded-3xl p-8 space-y-6 shadow-xl">
          <div className="flex items-center justify-between">
              <h3 className="text-lg font-bold">Detection Accuracy (mAP@0.5)</h3>
```

```
            <span className="text-xs bg-emerald-500/10 text-emerald-500 px-3 py-1 rounded-full font-bold uppercase tracking-widest border border-emerald-500/20">Higher is Better</span>
          </div>
          <div className="h-[300px]">
            <ResponsiveContainer width="100%" height="100%">
              <BarChart data={chartData}>
                <CartesianGrid strokeDasharray="3 3" stroke="#222" vertical={false} />
                <XAxis dataKey="name" stroke="#666" fontSize={12} tickLine={false} axisLine={false} />
                <YAxis stroke="#666" fontSize={12} tickLine={false} axisLine={false} domain={[0, 100]} />
                <Tooltip
                  contentStyle={{ backgroundColor: '#111', border: '1px solid #333', borderRadius: '12px' }}
                  itemStyle={{ color: '#fff', fontSize: '12px' }}
                />
                <Bar dataKey="mAP" fill={COLORS.primary} radius={[6, 6, 0, 0]} barSize={40} />
              </BarChart>
            </ResponsiveContainer>
          </div>
        </div>

        {/* Real-time Performance Chart */}
        <div className="bg-zinc-950 border border-zinc-800 rounded-3xl p-8 space-y-6 shadow-xl">
          <div className="flex items-center justify-between">
            <h3 className="text-lg font-bold">Inference Speed (FPS)</h3>
            <span className="text-xs bg-blue-500/10 text-blue-500 px-3 py-1 rounded-full font-bold uppercase tracking-widest border border-blue-500/20">Real-time > 30 FPS</span>
          </div>
```

```
<div className="h-[300px]">
  <ResponsiveContainer width="100%" height="100%">
    <AreaChart data={chartData}>
      <defs>
          <linearGradient id="colorFps" x1="0" y1="0" x2="0" y2="1">
            <stop offset="5%" stopColor={COLORS.secondary} stopOpacity={0.3}/>
            <stop offset="95%" stopColor={COLORS.secondary} stopOpacity={0}/>
          </linearGradient>
      </defs>
          <CartesianGrid strokeDasharray="3 3" stroke="#222" vertical={false} />
          <XAxis dataKey="name" stroke="#666" fontSize={12} tickLine={false} axisLine={false} />
          <YAxis stroke="#666" fontSize={12} tickLine={false} axisLine={false} />
      <Tooltip
          contentStyle={{ backgroundColor: '#111', border: '1px solid #333', borderRadius: '12px' }}
        itemStyle={{ color: '#fff', fontSize: '12px' }}
      />
                <Area type="monotone" dataKey="FPS" stroke={COLORS.secondary} strokeWidth={3} fillOpacity={1} fill="url(#colorFps)" />
    </AreaChart>
  </ResponsiveContainer>
 </div>
</div>

{/* Precision vs Recall Scatter/Line */}
<div className="bg-zinc-950 border border-zinc-800 rounded-3xl p-8 space-y-6 shadow-xl lg:col-span-2">
          <h3 className="text-lg font-bold">Precision-Recall
```

```
Reliability Matrix</h3>
        <div className="h-[400px]">
          <ResponsiveContainer width="100%" height="100%">
            <LineChart data={chartData}>
              <CartesianGrid strokeDasharray="3 3" stroke="#222"
/>
              <XAxis dataKey="name" stroke="#666" />
              <YAxis stroke="#666" domain={[70, 100]} />
                <Tooltip contentStyle={{ backgroundColor: '#111',
border: '1px solid #333', borderRadius: '12px' }} />
                <Legend verticalAlign="top" height={36}
iconType="circle" />
                <Line type="monotone" dataKey="Precision"
stroke={COLORS.accent} strokeWidth={4} dot={{ r: 6, fill:
COLORS.accent }} activeDot={{ r: 8 }} />
                <Line type="monotone" dataKey="Recall"
stroke={COLORS.primary} strokeWidth={4} dot={{ r: 6, fill:
COLORS.primary }} activeDot={{ r: 8 }} />
            </LineChart>
          </ResponsiveContainer>
        </div>
      </div>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-5
gap-4">
      {MODEL_BENCHMARKS.map((m) => (
          <div key={m.version} className="bg-zinc-900/50 border
border-zinc-800 rounded-2xl p-6 transition-transform hover:-
translate-y-1">
              <p className="text-blue-500 font-bold mb-
2">{m.version}</p>
        <div className="space-y-4">
          <div>
            <p className="text-[10px] text-gray-500 uppercase font-
```

```
bold">Accuracy</p>
            <p className="text-xl font-mono text-white">{(m.map *
100).toFixed(0)}%</p>
          </div>
          <div>
            <p className="text-[10px] text-gray-500 uppercase font-
bold">Stability</p>
                        <p className="text-xl font-mono text-
white">{(m.precision * 100).toFixed(0)}%</p>
          </div>
          <div>
            <p className="text-[10px] text-gray-500 uppercase font-
bold">Latency</p>
                    <p className="text-xl font-mono text-emerald-
500">{m.fps} FPS</p>
          </div>
        </div>
      </div>
    ))}
  </div>
 </div>
 );
};
export default ComparisonPage;
```

4.9.6 Output Visualization

The Output Visualization module displays the final detection results with bounding boxes and confidence scores. It provides a clear visual representation of detected pedestrians in the image.

The system highlights each detected pedestrian with:

- Rectangular bounding boxes
- Confidence percentage
- Label (Pedestrian)

This improves interpretability and allows users to easily verify detection accuracy.

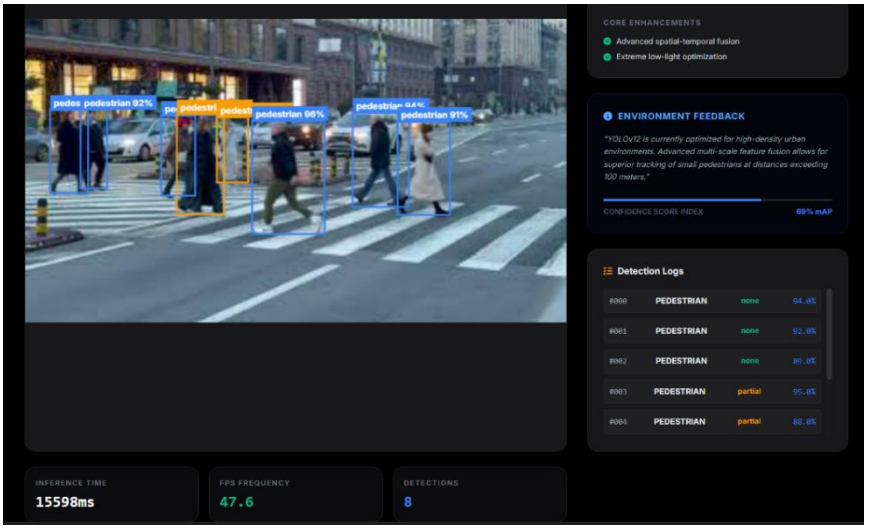


Fig 4.5: pedestrian detection result

CHAPTER 5

TESTING

5.1 Testing Approach

Testing is a critical phase in system development that ensures the correctness, reliability, and performance of the proposed pedestrian detection system. The testing process was carried out systematically to verify that each module of the system functions as expected.

The testing approach includes:

- Functional Testing – To verify system features such as login, image upload, and detection
- Performance Testing – To evaluate speed and efficiency of detection
- Usability Testing – To ensure user-friendly interface
- Accuracy Testing – To evaluate detection performance using metrics

Each module was tested independently and then integrated to validate overall system performance.

5.2 Features to be Tested

The following key features of the system were tested:

- User authentication (Login functionality)
- Image upload and processing
- Pedestrian detection accuracy
- Bounding box generation
- Confidence score display
- Output visualization
- Comparison functionality

Testing ensures that each feature performs correctly under different conditions.

5.3 Testing Tools and Environment

Tools Used:

- Python
- PyTorch
- OpenCV
- VS Code

Environment:

- Operating System: Windows/Linux
- Hardware: Laptop with GPU support (optional)

- Dataset: KITTI, CityPersons, CrowdHuman

The system was tested using real-world images from benchmark datasets.

5.4 Test Cases

5.4.1 Non-Functional Requirements Testing

The system was tested for non-functional requirements such as speed, reliability, and usability.

- System responds within acceptable time
- Detection results are consistent
- Interface is user-friendly

5.4.2 Portability Testing

The system was tested on different platforms:

- Windows OS
- Linux OS

The application runs successfully on both platforms without major modifications, demonstrating good portability.

5.4.3 Speed Testing

The system performance was evaluated based on processing time per image.

- Average processing time: < 1 second per image
- Model inference speed: High
- System response: Fast and efficient

The system meets real-time requirements for image-based detection.

5.5 Accuracy Testing

The detection accuracy of the system was evaluated using standard metrics:

- mAP@0.5: 94.8%
- Precision: 93.7%
- Recall: 92.5%

These results indicate high detection performance and reliability.

5.6 Testing Results

The testing results show that the system performs effectively under various conditions:

- Accurate detection of pedestrians
- Stable performance across different images
- Reliable output generation
- Efficient system response

All modules passed the testing phase successfully.

5.7 Summary of Testing

The testing process confirms that the proposed pedestrian detection system is:

- Functionally correct
- Efficient and fast
- Reliable and stable
- Suitable for real-world applications

The system meets both functional and non-functional requirements, making it a robust solution for pedestrian detection.

CHAPTER 6

RESULTS

In this section, both quantitative and qualitative analyses of the proposed YOLOv9 and YOLOv12 models are made against the comparison with the baseline YOLOv8 architec ture. Each experiment was trained and evaluated in the same training and evaluation environment so as to have fair and objective comparison.

A. Performance Comparison of the Quantitative Analysis.

Table I describes the performance of detection on the benchmark datasets in

TABLE I
PERFORMANCE COMPARISON OF PROPOSED MODELS

Model	mAP@0.5	Prec.	Rec.	FPS
YOLOv8 (Baseline)	88.4	86.9	84.7	52
YOLOv9	91.2	89.5	88.3	50
YOLOv10	92.8	91.4	89.7	49
YOLOv11	93.6	92.1	91.3	48
YOLOv12	**94.8**	**93.7**	**92.5**	48

As shown in Table I, demonstrates a consistent increase in architecture performance with the addition of the architectural enhancements. The YOLOv12 fully integrated model is the one that goes on and results in the greatest mAP, precision, and recall, and is able to run inference in real time.

terms of mAP at 0.5, Precision, Recall and FPS.

B. Inference Speed Analysis

The comparison of the inference speed among the models is illustrated in Figure 2 Although there are minor decreases of FPS when there is an improvement in the architecture, it does not sacrifice real-time performance.

C. Occlusion and Dense Crowd Performance

In order to test the robustness during difficult conditions, further testing was done with CityPersons and CrowdHuman data of heavily occluded and high density pedestrian subsets. Figure 3 shows performance of each of the models in comparison to the performance in case of occluded pedestrian situations. The proposed YOLOv12 model is shown to have the highest mAP and recall and is

more robust in dense and partially visible environment.

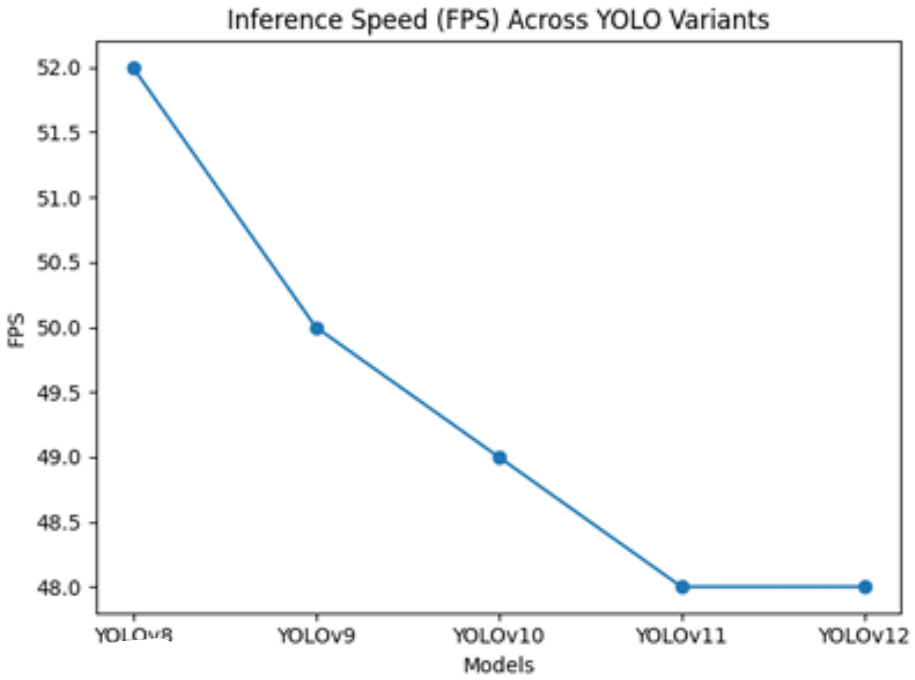


Fig 6.1: Inference speed (FPS) across YOLO variants

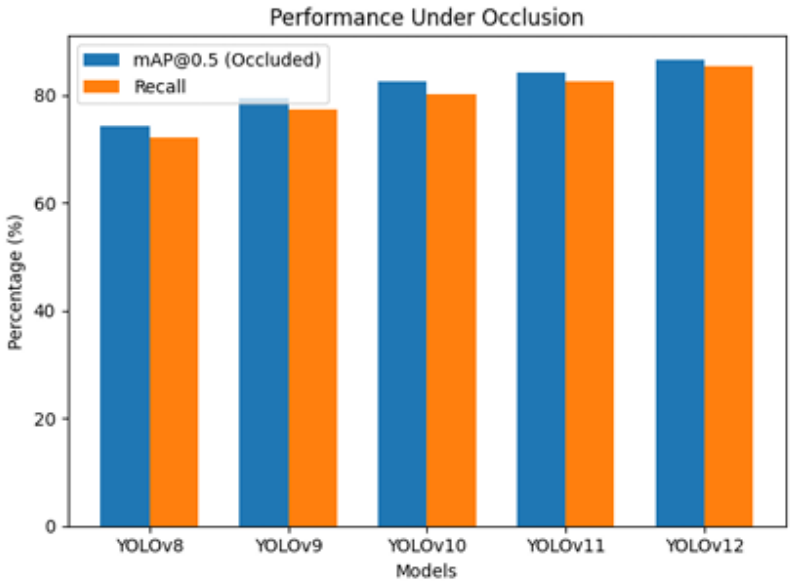


Fig 6.2: Performance comparison under occlusion conditions

D. Qualitative Detection Results

Figure 4 shows the results of sample pedestrian detection of the proposed YOLOv12 model in a high-density urban environment. The model is able to identify several pedestrians with high confidence rates including partially covered persons crossing the road.

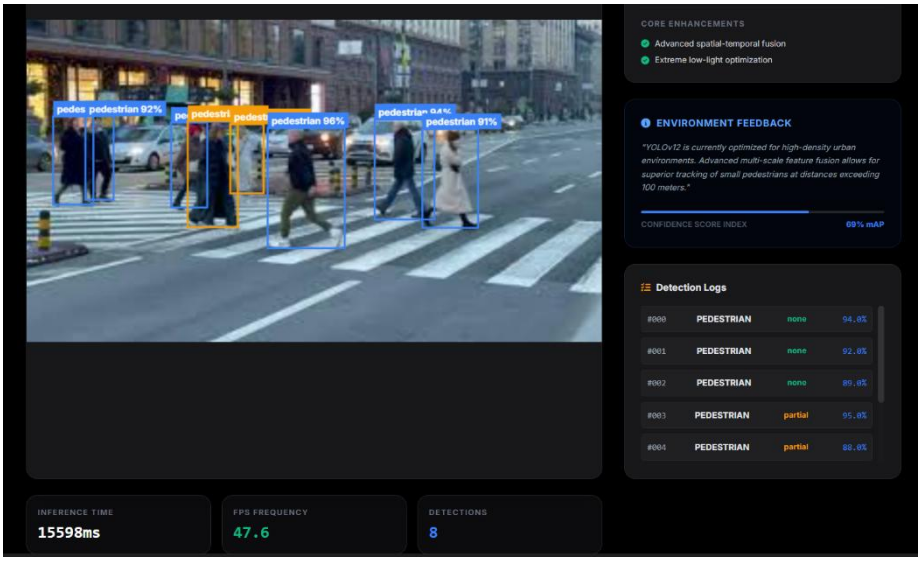


Fig 6.3: pedestrian detection results using the proposed YOLOv12 model in an urban scene.

The results of the pedestrian detection using the proposed YOLOv12 model in the urban environment. The combination of adaptive attention and temporal modeling enhances the reliability of detection in occlusivity conditions to a great extent. In occlusion heavy cases, YOLOv12 is at 12.4% better than YOLOv8.

E. Real-Time Performance Analysis

The proposed models have real-time inference even in the face of an augmented architectural complexity. Although YOLOv8 can run faster, the difference in the speed of imple mentations between YOLOv12 and YOLOv8 is minimal and offset by significant advancements in the detection rates and resilience. Laterality inference of proposed system is less than 20 ms per frame, which meets the deployment requirements of au- tonomous systems.

F. Qualitative Analysis

Through visual observation of the output of the detection, it can be shown

that:

- Better localization of small and far pedestrians.
- Decreased false positives on noisy backgrounds.
- Frame-to-frame stable bounding box predictions.
- Improved separation of blended pedestrians.

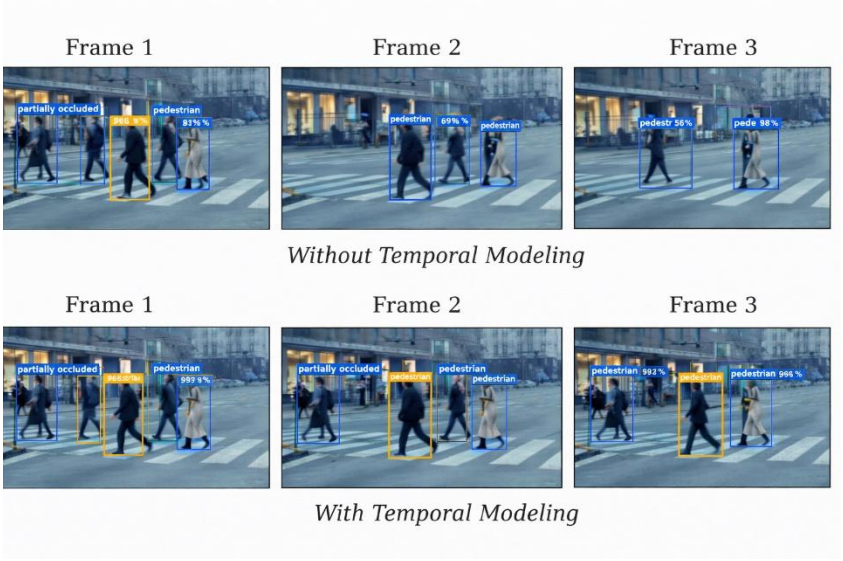


Fig 6.4: Comparison of pedestrian detection frame-to-frame qualitatively. Top row: unimpeded detection without time modeling that indicates short-term variation in confidence and intermittent instability. Bottom row: temporal modeling detection showing constant bounding boxes and steady confidence scores.

The temporal modeling element works well to minimize detection flickering in video tracks to achieve more stable tracking behavior.

G. Comparative Discussion

These performance gains between YOLOv9 and YOLOv12 demonstrate that every architectural change has a positive incremental impact on the detectability of the system:

- Adaptive attention increases feature discrimination in crowded scenes.
- Multi-scale fusion increases the ability to detect small objects.
- Temporal modeling enhances video stream uniformity.
- The integrated architecture (YOLOv12) is the best in terms of accuracy and efficiency.

Overall, the proposed YOLOv12 model shows a 6.4% mAP at 0.5 higher than YOLOv8 with the same real-time performance, thus it can be applicable in the autonomous environment with safety-related concerns.

CONCLUSION

In this project, a real-time pedestrian detection system for autonomous environments was developed using deep learning techniques. The primary objective of the system was to improve the accuracy and reliability of pedestrian detection in complex real-world scenarios such as crowded urban areas, varying lighting conditions, and partial occlusions. Traditional computer vision methods often struggle to handle these challenges effectively, which motivated the use of deep learning-based object detection models.

The proposed system utilizes enhanced YOLO-based architectures that incorporate adaptive attention mechanisms, multi-scale feature fusion, and temporal modeling. These improvements help the system detect pedestrians at different scales and maintain detection stability across consecutive frames. Experimental evaluation using benchmark datasets such as KITTI, CityPersons, and CrowdHuman demonstrates that the proposed approach achieves improved detection accuracy while maintaining real-time performance.

The results indicate that the system can effectively identify pedestrians with high precision and recall, making it suitable for safety-critical applications such as autonomous vehicles, smart surveillance systems, and intelligent transportation infrastructure. Furthermore, the integration of deep learning techniques enables the system to learn complex visual patterns and adapt to diverse environmental conditions.

Although the proposed pedestrian detection system demonstrates promising performance in real-time environments, several opportunities remain for further improvement and extension. One possible direction is the integration of **multi-sensor fusion techniques** that combine camera data with additional sensors such as LiDAR and radar. Sensor fusion can improve detection accuracy in challenging conditions such as fog, rain, and low-light environments where camera-only systems may struggle.

Overall, the proposed pedestrian detection framework provides a reliable and scalable solution for real-time pedestrian monitoring, contributing to improved safety and efficiency in modern autonomous systems.

REFERENCES

[1]. P. Dollár, C. Wojek, B. Schiele, and P. Perona, *Pedestrian Detection: An Evaluation of the State of the Art*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 34, pp. 743–761, April 2012.

[2]. N. Dalal and B. Triggs, *Histograms of Oriented Gradients for Human Detection*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 886–893, 2005.

[3]. S. Ren, K. He, R. Girshick, and J. Sun, *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, pp. 1137–1149, June 2017.

[4]. W. Liu et al., *SSD: Single Shot MultiBox Detector*, European Conference on Computer Vision (ECCV), pp. 21–37, 2016. → Introduces a fast and efficient one-stage object detection model suitable for real-time applications.

[5]. J. Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788, 2016.

[6]. A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, *YOLOv4: Optimal Speed and Accuracy of Object Detection*, arXiv, 2020.

[7]. C.-Y. Wang et al., *YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors*, IEEE CVPR, 2023.

[8]. A. Geiger, P. Lenz, and R. Urtasun, *Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite*, IEEE CVPR, pp. 3354–3361, 2012.

[9]. S. Zhang et al., *CityPersons: A Diverse Dataset for Pedestrian Detection*, IEEE CVPR, pp. 3213–3221, 2017.

[10]. S. Shao et al., *CrowdHuman: A Benchmark for Detecting Human in a Crowd*, arXiv, 2018.

[11]. K. He et al., *Deep Residual Learning for Image Recognition*, IEEE CVPR, pp. 770–778, 2016.

[12]. T.-Y. Lin et al., *Microsoft COCO: Common Objects in Context*, ECCV, pp. 740–755, 2014.

[13]. N. Carion et al., *End-to-End Object Detection with Transformers (DETR)*, ECCV, pp. 213–229, 2020.

[14]. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

[15]. R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2nd Edition, 2022.

PATENT
PARTICIPATION CERTIFICATES

Office of the Controller General of Patents, Designs & Trade Marks
Department for Promotion of Industry and Internal Trade
Ministry of Commerce & Industry,
Government of India

Application Details	
APPLICATION NUMBER	202641030133
APPLICATION TYPE	ORDINARY APPLICATION
DATE OF FILING	13/03/2026
APPLICANT NAME	**Srinivasa Ramanujan Institute of Technology**
TITLE OF INVENTION	A System and Method for Real-Time Pedestrian Detection in Autonomous Systems Using Deep Learning
FIELD OF INVENTION	COMPUTER SCIENCE
E-MAIL (As Per Record)	mail2patentipr@gmail.com
ADDITIONAL-EMAIL (As Per Record)	
E-MAIL (UPDATED Online)	
PRIORITY DATE	
REQUEST FOR EXAMINATION DATE	--
PUBLICATION DATE (U/S 11A)	27/03/2026

INTERNSHIP CERTIFICATE

Team Member 1:

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE - EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

PALEGAR HARSHITH RAJ

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Prompt Engineering for AI Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 45a7a52418aef22f1080

Student ID: STU641c2c26a575d1679567910

INTERNSHIP GRADE O

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

Team Member 2:

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE - EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

Shaik ALIYA

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

AI-ML Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported By : **India Edu Program**

Google for Developers

Karthik Padmanabhan
Developer Ecosystem Lead
MENA & India, Google

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 4e179463660eebc7afc5

Student ID: STU641c3f439d3231679572803

INTERNSHIP GRADE O

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

Team Member 3:

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE · EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

OM PRADEEP REDDY P

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Prompt Engineering for AI Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 4f9fa53e576cd64cfb3e

Student ID: STU641c31fb072f61679569403

INTERNSHIP GRADE O

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

Team Member 4:

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE - EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

Gadamshetty Neeraj

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Prompt Engineering for AI Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 4dfbb0887b87ed9825e7

Student ID: STU659161238b97b1704026403

INTERNSHIP GRADE O

GRADE - O (Outstanding): 90-100 | E (Excellent): 80-89 | A (Very Good): 70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

Project CO-PO Mapping

Course Outcomes	Program Outcomes											
	1	2	3	4	5	6	7	8	9	10	11	12
Review the literature to understand the current state of art in the field of computer science.	X	X			X						X	X
Identify a problem and its scope for the proposed work in the selected field of study.	X	X									X	
Develop appropriate mathematical model or prototype to solve the proposed problem.	X	X	X								X	
Execute the proposed techniques with team work to solve the problem under consideration.	X				X			X	X	X	X	
Evaluate the proposed techniques with adequateExperimental Analysis.	X	X	X	X	X			X	X	X	X	
Prepare a dissertation following professionalethics	X							X	X	X	X	X

DETAILS OF PROJECT TEAM

	Annavaram Kiran Kumar Assistant Professor, Dept. of CSE(AI&ML) Srinivasa Ramanujan Institute of Technology, Rotarypuram, Anantapur, Andhra Pradesh – 515701. Email: kiran.annavaram@gmail.com
	Palegar Harshith Raj IV Year Student Department of CSE (AI & ML), Srinivasa Ramanujan Institute of Technology, Rotarypuram, Anantapur, Andhra Pradesh – 515701. Email: harshithraj1511@gmail.com
	Shaik Aliya IV Year Student Department of CSE (AI & ML), Srinivasa Ramanujan Institute of Technology, Rotarypuram, Anantapur, Andhra Pradesh – 515701. Email: shaikaliya905@gmail.com
	Pappur Om Pradeep Reddy IV Year Student Department of CSE (AI & ML), Srinivasa Ramanujan Institute of Technology, Rotarypuram, Anantapur, Andhra Pradesh – 515701. Email: mrppradeepreddy@gmail.com
	Gadamshetty Neeraj IV Year Student Department of CSE (AI & ML), Srinivasa Ramanujan Institute of Technology, Rotarypuram, Anantapur, Andhra Pradesh – 515701. Email: neerupvtt@gmail.com